

URL Shortener - URL Redirecting Deep Dive

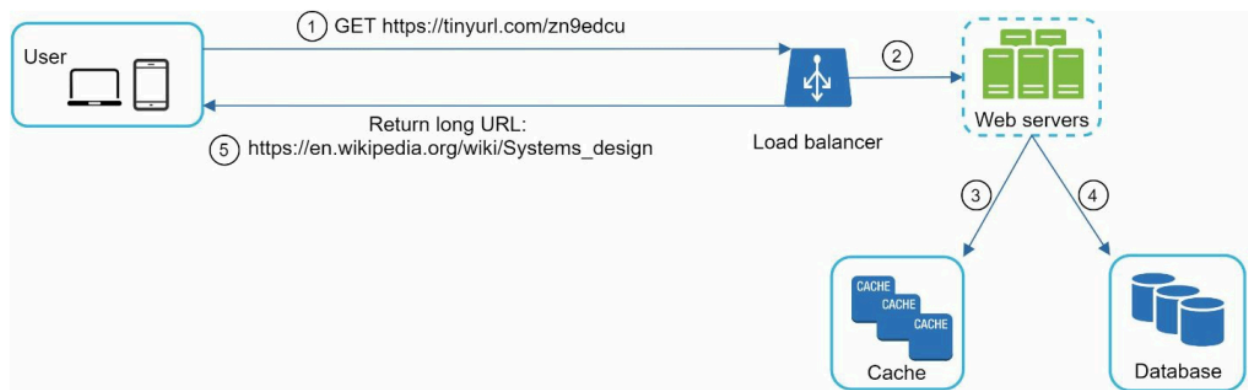
1. Overview

URL redirecting is a **read-heavy operation**.

Key Idea:

- Store `<shortURL, longURL>` mapping in **cache**
- Reduce database access
- Improve latency and scalability

2. High-Level Flow



3. Step-by-Step Flow

Step 1: User Request

None

User clicks:

<https://tinyurl.com/zn9edcu>

Step 2: Load Balancer

None

Request → Load Balancer → Web Server

Distributes incoming traffic across multiple servers

Step 3: Cache Lookup

None

Check in Cache:

shortURL → longURL

Case 1: Cache Hit

If found in cache:

None

Return longURL immediately

- ✓ Fast response
- ✓ No database access

Case 2: Cache Miss

If NOT found:

None

Query Database

Step 4: Database Lookup

None

```
longURL = DB.get(shortURL)
```

Invalid URL Case

If not found in DB:

None

Return error (invalid shortURL)

Step 5: Update Cache

If found in DB:

None

Store mapping in cache:

shortURL → longURL

✓ Improves future performance

Step 6: Redirect User

None

Return longURL

→ Perform HTTP redirect (301 or 302)

4. Why Cache is Important

Because:

- Read operations >> Write operations
- Cache provides:
 - Low latency
 - Reduced DB load
 - High throughput

5. Key Components

Load Balancer

- Distributes traffic
- Improves availability

Cache (e.g., Redis)

- Stores hot data
- Handles majority of requests

Database

- Source of truth
- Stores persistent mapping

6. Key Takeaways

- Cache-first strategy is critical for performance
- DB is fallback when cache misses
- System is optimized for **read-heavy workload**
- Invalid short URLs handled gracefully